Situation engineering: separating evidence applicability from availability in AI systems

Geunsik Lim[1*]

[1] College of Information and Communication Engineering, Sungkyunkwan University, Suwon, South Korea .

Corresponding author(s). E-mail(s): leemgs@g.skku.edu;

Abstract

Artificial-intelligence systems can retrieve a well-supported fact yet apply it to the wrong time, scope, epistemic state or possible world. We call this failure *situation blindness* and define *situation engineering* as constructing the query-relative state that governs evidence applicability. We introduce a state interface, Situation-Catching Question Answering, and SituationCatch-Bench, a 4,200-item diagnostic spanning temporal change, missing premises, source conflict, modality, scope, observer knowledge and counterfactual worlds. A deterministic implementation satisfies every invariant under oracle states; accuracy falls to 77.9% under text-inferred states and 35.1% after corruption, and matched ablations produce category-specific failures. These results establish an executable diagnostic, not language-model superiority. In a preliminary single-family test (three Gemini models, $\boldsymbol{n = 210}$), explicit situation prompting does not beat matched structured prompting and trails focused retrieval, leading only on temporal and observer items; these category comparisons are exploratory. This bounded result is consistent with applicability *sensing*, not a prompt slot, being the bottleneck.

Keywords: large language models, situation engineering, evidence applicability, situation awareness, hallucination, trustworthy artificial intelligence, temporal reasoning, question answering, agent systems

Large language models (LLMs) increasingly answer questions by combining parametric knowledge with retrieved documents, tools, memory and long conversational histories. These mechanisms improve evidence coverage, but they do not guarantee that a system identifies the *situation* in which a fact is valid. A model may read that a person was appointed and later resigned, yet still name that person as the current office holder.

It may turn a proposal into a decision, extend a local policy to an entire organization, answer from information unavailable to the queried observer, or collapse a hypothetical scenario into the real world. In each case the decisive evidence can already be present. The failure is not simply missing knowledge; it is incorrect evidence applicability.

We define *situation blindness* as producing an answer or action that is supported by at least one available claim but inconsistent with the time, scope, modality, observer knowledge or possible world required by the query. This category matters because conventional factuality checks can mark the supporting claim as true while missing that it is true in the wrong situation. Natural question-answering resources already show that a material subset of ordinary questions depends on extra-linguistic time or place, and that updated evidence does not eliminate the resulting performance loss [1–5].

Retrieval-augmented generation supplies external evidence [6]; self-reflective and tool-using systems add critique and action [7–9]; context engineering organizes the inference-time information payload [10]; and agent harnesses mediate tools, memory, permissions, verification and feedback [11, 12]. These layers are necessary, but none requires an explicit answer to a separate question: *which interpretation of the available evidence governs this interaction now?*

We call the corresponding discipline *situation engineering*: the systematic design of representations, sensing interfaces, update rules, validation procedures and response policies that establish the active query-relative world state before an AI system commits to an answer or action. For a query q and evidence E, a minimal state is

$$S(q, E) = \{\mathcal{A}, \mathcal{V}, \mathcal{T}, \mathcal{P}, \mathcal{M}, \mathcal{K}, \mathcal{W}, \mathcal{U}\}, \tag{1}$$

where the variables denote actors, events, valid time, scope, modality and source status, observer-specific knowledge, possible world and unresolved answer-critical information. A policy then selects

$$\pi(S) \in \{\textsc{Answer}, \textsc{Clarify}, \textsc{Retrieve}, \textsc{Abstain}, \textsc{Act}\}. \tag{2}$$

This decomposition separates evidence availability from evidence applicability.

The present paper makes four contributions. First, it defines situation blindness as a testable failure class and relates it to established work on situations, temporal reasoning, belief state tracking and knowledge revision. Second, it specifies situation engineering as a systems contract connecting prompt, context, retrieval, memory, tool, harness, evaluation and serving engineering without subsuming them. Third, it introduces SCQA and SituationCatch-Bench, and uses matched state conditions and component ablations to demonstrate that the specified applicability mechanisms are executable and causally separable. Fourth, it reports an audited, deliberately bounded three-model Gemini-family intervention with a focused retrieval baseline. Its null-to-negative aggregate result prevents us from equating an explicit prompt slot with situation awareness, while its temporal and observer-category pattern motivates a larger cross-family natural-text test. This claim boundary distinguishes what we demonstrate from broad language-model generalization.

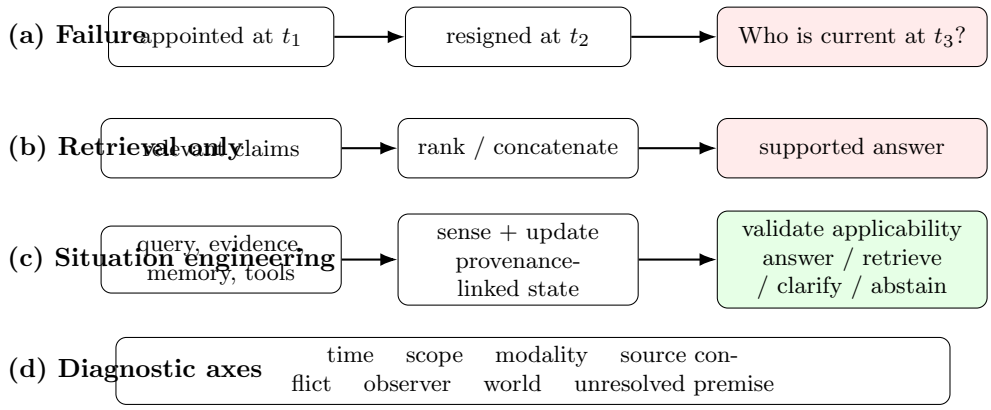


Fig. 1 From situation blindness to an applicability-aware response. **a**, A supported appointment becomes stale after resignation. **b**, retrieval and relevance alone do not require resolution of that transition. **c**, situation engineering constructs a provenance-linked state, validates claim applicability and propagates unresolved variables into response policy. **d**, SituationCatch-Bench isolates seven corresponding diagnostic axes.

Situation Engineering framework

Prompt engineering specifies a task; context and retrieval engineering select information; memory engineering retains observations; tools expose capabilities; and harness engineering controls execution. Situation engineering addresses a narrower semantic question: which available claims apply to this query, observer and world now? It is therefore a proposed interface among existing components, not a replacement name for them and not an empirically validated universal layer. Figure 1 contrasts situation blindness, retrieval-only processing and an applicability-aware response, and summarises the seven diagnostic axes that SituationCatch-Bench isolates.

For query q, evidence E, retained memory M_t, tool observations O_t and harness trace H_t, situation construction produces candidate states $\mathcal{S}_t$ with unresolved variables U_t and provenance P_t:

$$(q, E, M_t, O_t, H_t) \rightarrow (\mathcal{S}_t, U_t, P_t). \tag{3}$$

Each evidence claim is assigned a query-relative applicability value

$$\mathrm{Apply}(e, q, S_t) \in \{\mathrm{valid}, \mathrm{invalid}, \mathrm{unresolved}\}. \tag{4}$$

Preserving *unresolved* is essential. A system should retrieve, clarify or abstain when answer-critical state cannot be established rather than silently converting uncertainty into a plausible binary judgement.

Operations and component boundaries

Six operations define the interface. *Sensing* extracts actors, events, times, modality, source status, scope, observer knowledge and world identity. *Assembly* links

observations into candidate states. *Updating* applies supersession, cancellation and belief transitions. *Validation* checks applicability, conflicts, provenance and missing variables. *Policy* selects an answer, retrieval, clarification, abstention or action. *Observability* records the active state and its evidence.

These responsibilities do not absorb neighbouring disciplines. A retriever still owns ranking and freshness; a memory system owns retention; a tool layer owns invocation semantics; a harness owns permissions and recovery; and serving systems own resource efficiency. The proposed situation layer consumes their outputs and returns applicability decisions and unresolved-state signals. Likewise, longer context is not sufficient: it may contain an appointment and a later resignation, a proposal and a final decision, or real and hypothetical claims simultaneously. Applicability requires resolving their relation rather than merely retaining both.

SCQA as a bounded control implementation

Situation-Catching Question Answering (SCQA) instantiates the interface for seven controlled variables. Its deterministic sensor or supplied gold state feeds auditable update rules and an answer/clarify/abstain policy. SCQA has no learned parameters. It is designed to test whether removing a required variable produces the matching failure signature, not to compete with contemporary language models.

This scope matters for interpretation. The generated benchmark and solver share an ontology, making the oracle-state experiment analogous to a protocol conformance suite. The predicted-state condition introduces sensing error but still uses generated English. Natural-language generalization, comparison with strong prompting and retrieval systems, multilingual transfer, human annotation and deployment efficiency remain untested hypotheses. The released multi-provider and annotation programs make these studies executable but do not turn an unexecuted protocol into evidence.

Testable predictions

The framework yields falsifiable predictions for future matched experiments. Holding model, evidence and computation budget fixed, an explicit situation state should improve the categories whose decisive variables are otherwise implicit. Gains should diminish when a baseline already represents the same state. Predicted-state errors should decompose into sensing, update, policy and generation failures. Corrupting one state slot should selectively damage its matching category. Finally, the benefit must be evaluated jointly with token use, latency, retrieval calls, monetary cost and unnecessary clarification. Failure of these predictions would argue against treating situation engineering as a distinct practical interface.

Results

A controlled benchmark isolates seven applicability invariants

SituationCatch-Bench v0.1 contains 4,200 generated English instances, with 600 instances in each of seven diagnostic categories: temporal state change, omitted answer-critical premises, source conflict, proposed versus confirmed events, local versus

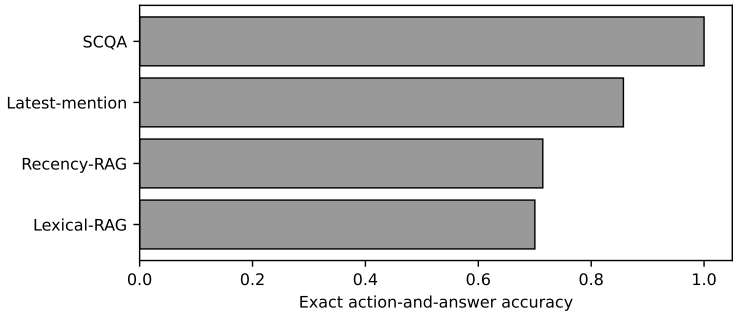


Fig. 2 Oracle-state diagnostic accuracy. The result demonstrates that the specified invariants are executable. It does not estimate end-to-end LLM accuracy because situation variables are supplied rather than inferred.

universal scope, observer-dependent knowledge and real versus counterfactual worlds. Every item provides a question, shuffled claims, query time, required state variables, a gold action and, where answerable, a gold answer. The benchmark is intentionally transparent: each category is a software-level test of one applicability invariant rather than a claim to natural linguistic diversity.

Oracle-state control satisfies the generated invariants

SCQA was compared with lexical retrieval, recency-ranked retrieval and a latest-mention rule. All methods received the same generated claims; SCQA alone used the complete oracle situation state and transition semantics. SCQA obtained 100.0% action-and-answer accuracy. Latest mention obtained 85.7%, recency ranking 71.4% and lexical retrieval 70.0% (Fig. 2). These numbers are deterministic consequences of the benchmark and implementation and should be interpreted as regression-test coverage, not as estimates of model performance in a population of natural questions.

The 85.7% value is structurally induced: the categories are equally weighted and the latest-mention control systematically fails the hidden-premise category, so success on six categories yields $6/7 = 85.7\%$. The 14.3 percentage-point gap is therefore not presented as an empirical effect size against a competitive baseline.

The category-level pattern provides the useful result. Latest mention fails on hidden-premise items because it answers when clarification is required. Recency ranking fails when the newest global event is not known to the queried observer. Lexical overlap is vulnerable to weak but topically similar conflicting claims. Thus a single aggregate "hallucination" label hides distinct and testable mechanisms (Fig. 3).

Component ablations produce matched failure signatures

Removing temporal validity, source status, modality, scope, observer knowledge, world identity or missing-variable detection selectively damages the matching category (Fig. 4). This causal alignment is the main empirical contribution of the controlled study. It shows that the implementation does not merely benefit from a generic instruction to be cautious; behaviour changes when the specific variable required by an item

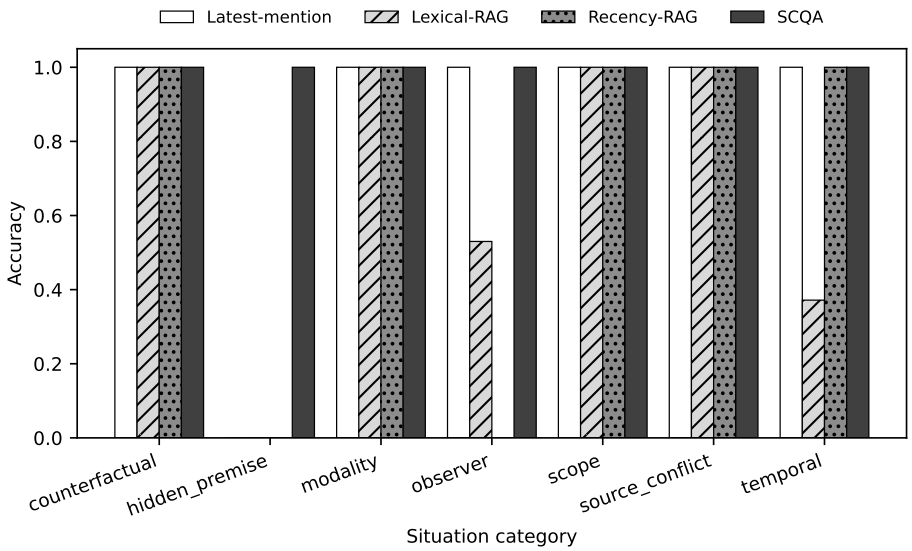


Fig. 3 Accuracy by diagnostic category. Each category isolates one applicability variable, enabling mechanism-level failure localization.

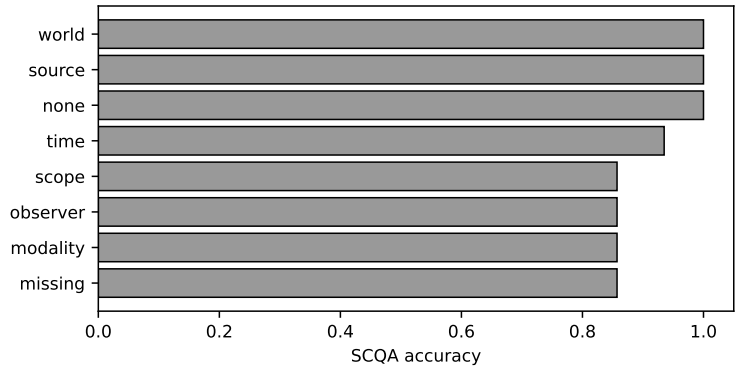


Fig. 4 Oracle-state component ablations. Each removed variable recovers the predicted category-specific failure, providing a causal software diagnostic.

is removed. Because benchmark and solver share the same schema, the result is best viewed as a mechanistic unit test that future neural systems should pass under both gold and predicted states.

Partial observability exposes a sensing bottleneck

We evaluated two observability regimes. In the evidence-complete control, all required variables are supplied in the structured state. In the text-only regime, the sensor must reconstruct a state from the rendered claims; variables not expressed in those claims remain unobservable rather than being silently copied from gold metadata.

Across all 4,200 diagnostic instances, action-and-answer accuracy fell from 100.0% with gold states to 77.9% with predicted states. Deliberately overwriting source status, modality and scope in the predicted state reduced accuracy further to 35.1%. These are complete executions, not projected values. They show that the oracle result is not preserved when situation sensing is imperfect. Because the texts are generated from the same seven scenario families and the sensor is a transparent rule baseline, these results remain controlled diagnostics rather than evidence of natural-language or LLM generalization.

This comparison is therefore a partial-observability stress test, not a fair estimate of extraction from evidence-complete prose. In particular, a missing time should induce retrieval, clarification or abstention when it is answer-critical; it should not be scored as though a sensor could infer an unwritten value. A confirmatory natural-text study must consequently report separate text-observable and answer-critical-missing strata.

Claim-level scoring localizes the sensor's limitations (Extended Data Table 1). Event type reached 92.3%, modality 100.0%, scope 61.5%, actor 30.8% and joint source/status 30.8%. Temporal expressions and validity intervals scored 0% because the generated claim strings omit the numeric times stored in the hidden schema: a system cannot reconstruct a state variable that is neither expressed nor retrieved. Observer and world slots reached 92.3% and 100.0% under repetitive generated language and are not estimates of natural extraction. In this deterministic pipeline, 927 final errors are attributable to sensing, zero to update/policy under gold state, and generation is a fixed renderer.

Progressive stress testing separates execution, sensing and validation

Read as a three-stage stress test, the complete benchmark produces 100.0% $\rightarrow$ 77.9% $\rightarrow$ 35.1%: an oracle state tests execution, a text-derived state adds sensing error, and targeted corruption tests whether unvalidated state can propagate into the answer. The corresponding losses are 22.1 and 42.8 percentage points. This progression is empirical in the limited sense that every stage was executed on all 4,200 items and item-level outputs are archived. It is not a learning curve, a comparison of three models or an estimate of robustness to naturally occurring corruption. In particular, the last stage is an intentionally severe intervention selected to damage source, modality and scope fields; monotonic degradation is therefore a diagnostic expectation rather than an independently discovered population effect.

This decomposition also prevents the oracle result from carrying the paper's neural claim. The E5 test below is the relevant prompt-level experiment, and its overall result is null-to-negative within one provider family. Accordingly, the combined evidence supports a sensing-and-validation bottleneck, but not the claim that the present situation prompt solves it.

Table 1 Evidence ladder and permitted interpretation.

Evidence level	What is available	Permitted claim
E1: formal definition	Situation schema and applicability relation	The failure class is operationally testable
E2: oracle unit tests	4,200 generated instances and matched ablations	Specified invariants are executable and causally separable
E3: natural prevalence	Independent temporal/situated QA literature	Situation dependence occurs in natural questions
E4: predicted-state control	Deterministic text sensor on 4,200 generated items	Sensing errors reduce controlled end-to-end accuracy; natural-text generalization remains open
E5: multi-model intervention	Audited single-family (Gemini) pilot, $n = 210$, situation/structured/RAG/agent conditions	Situation prompting does not improve on structured prompting overall and trails focused retrieval; temporal and observer category differences are exploratory; multi-family and natural-text confirmation remain open
E6: deployment / human study	Not completed	Needed for operational safety and human comparison

Natural-language benchmarks establish prevalence, not SCQA efficacy

Independent resources show that situation dependence is not confined to the generated benchmark. SituatedQA reported that approximately 16.5% of sampled NQ-Open questions depended on time or place and that updated evidence did not remove a substantial present-time performance gap [1]. FreshQA, PAT-Questions, TimeQA, ChronoQA and TDBench test related problems in freshness, evolving facts and temporal validity [2–5, 13]. These studies motivate the problem definition, but they are not evaluations of SCQA and are not counted as evidence that the proposed intervention improves a frontier model.

An evidence ladder separates established findings from open claims

Table 1 makes the claim boundary explicit. The current study shows within its co-designed conformance suite that seven applicability invariants can be represented, executed and independently ablated under oracle structured states. The audited E5 pilot tests, but does not establish, the hypothesis that explicit situation states reduce analogous errors in LLMs: the prompt intervention is null-to-negative overall in one provider family, while temporal and observer items show a category-specific advantage over focused retrieval. Broad end-to-end efficacy still requires independent model families, a larger natural corpus, agent baselines and human annotation. We release raw responses, the adapter, schemas and reporting protocol and do not substitute unexecuted predictions for results.

Table 2 Gemini-family E5 accuracy with 95% cluster-bootstrap confidence intervals (categories as clusters; 3000 resamples), the paired situation−structured contrast, and cost. CIs that exclude zero are flagged as directional estimates only within this bounded analysis.

Model	Direct	Structured	Situation	$\Delta_{\text{sit}-\text{str}}$	Sit. tokens
2.5 Flash	83.3	81.9	83.3	+1.4	311
95% CI	[54.3, 100.0]	[52.9, 100.0]	[61.0, 100.0]	[−16.2, +17.1]	
2.5 Flash Lite	66.7	82.9	72.9	−10.0	270
95% CI	[31.4, 94.8]	[53.3, 100.0]	[46.7, 91.9]	[−19.5, −1.9]	
3.5 Flash	86.7	85.7	85.7	+0.0	285
95% CI	[59.0, 100.0]	[57.1, 100.0]	[57.1, 100.0]	[+0.0, +0.0]	

A preliminary matched multi-model intervention (E5)

The controlled results establish that the seven applicability invariants are executable and causally separable (E1–E4); they do not show that *prompting* a language model for an explicit situation state improves it. As a first, deliberately bounded test of that E5 claim, three Gemini models answer a stratified, evidence-complete sample of SituationCatch-Bench (30 items per category, $n = 210$) with evidence, decoding and budget held fixed, varying only the prompt: a plain *direct* answer, a *structured* JSON prompt or an explicit *situation*-state prompt (Methods). The primary quantity is the within-model paired situation−structured contrast; the run passes a completion audit with no failed calls.

The result is null-to-negative (Table 2). Prompting for an explicit situation state never yields a cluster-bootstrap interval indicating an improvement over a matched structured prompt: the paired situation−structured differences are +1.4, −10.0 and +0.0 percentage points, with a 95% cluster-bootstrap interval that excludes zero only for `gemini-2.5-flash-lite`, where the situation prompt is *worse* under this analysis. The intervals are otherwise wide because the benchmark has only seven template families, so resampling categories – the honest unit of independence – leaves little power for a directional claim. The situation and structured conditions also use roughly three times the total provider-reported tokens and have substantially higher median latency than the direct prompt, with no accuracy gain and no reduction in selective risk.

A small natural-language check points the same way. On a 12-item sample from the public SituatedQA temporal split, the two date-anchored conditions show no advantage for the situation prompt over a structured one (descriptive differences of −16.7, +0.0 and +0.0 points across the three models). At $n = 12$ this is a smoke test, not a result.

A retrieval-augmented baseline sharpens the picture (Extended Data Table 2). Answering from only the top-3 lexically retrieved claims, without situation metadata, is the strongest condition overall (91.3% versus 83.5% structured and 80.6% situation): a focused, relevant subset beats all claims plus a state instruction. Descriptively, the exception is the two categories that most require reasoning over the applicability state – temporal validity and observer knowledge – where the full-evidence situation condition beats RAG (+8.9 and +5.6 points); given category-level multiplicity this

pattern is exploratory, not confirmatory. Where applicability depends on which claim is valid *now* or is known to *this* observer, retrieving the most similar text is not enough. A tool-using agent that issues its own searches over the item's claims does not close this gap: it is the weakest condition overall (77.5%) and drops furthest on temporal (64.4%) and observer (66.7%) items, because self-directed retrieval can still fetch the most similar claim rather than the applicable one.

We read this as motivating the paper's distinction rather than as a plug-in prompting win: a prompt-level "situation" field is not situation awareness. Consistent with the controlled finding that accuracy collapses once states must be *sensed* from text (100.0% $\rightarrow$ 77.9%), the bottleneck is situation sensing and validation, not a state slot in the prompt. This single-provider, $n = 210$ pilot neither confirms nor refutes the hypothesis at scale; a confirmatory E5 requires multiple model families, a larger natural-language set and human agreement (Methods).

Discussion

The core insight is that relevance is not applicability. Retrieval can return a well-supported statement while a system still applies it to the wrong time, scope, epistemic state or possible world. Situation engineering turns this failure from an informal complaint about "missing context" into a set of state variables, update rules and behavioural consequences that can be tested separately.

A complementary layer, not a replacement vocabulary

Prompt, context, retrieval, memory, tool, harness, evaluation, security and serving engineering retain distinct responsibilities. Prompts specify the task; context and retrieval supply evidence; memory preserves observations; tools provide capabilities; harnesses govern workflow and permissions; evaluations measure behaviour; serving systems manage resource constraints. Situation engineering owns the query-relative semantics of the active state and the propagation of unresolved applicability into policy. A useful contract is

$$(C_t, M_t, O_t, H_t) \rightarrow (\mathcal{S}_t, U_t, P_t), \tag{5}$$

where context, memory, tool observations and harness traces produce candidate situations $\mathcal{S}_t$, unresolved variables U_t and the permitted policy P_t.

This contract prevents conceptual inflation. Retrieval quality should not be renamed situation engineering, and a JSON field named "situation" is not evidence of situation awareness. A system must show that state representation causally changes behaviour in predicted cases, that state uncertainty affects answering or action, and that failures can be assigned to sensing, update, validation, policy or generation.

Implications for hallucination and agent evaluation

A generated claim can be textually entailed by a document yet wrong for the query. Evaluation should therefore record both support and applicability. In free-form generation, claims should be linked to the situation state and provenance on which they

depend. In agents, tools should expose preconditions, state transitions and failure semantics rather than only free-text outputs; harnesses should block consequential actions while answer-critical state is unresolved. Evaluation engineering becomes the sensor layer that checks each transition and not merely the final answer.

The response policy is also part of correctness. Clarification and abstention are not failures when the situation is under-specified. Conversely, excessive clarification is costly and can make systems unusable. Future evaluations should therefore report answer accuracy, clarification utility, unnecessary-question rate, abstention precision, selective risk, coverage, latency, token use and monetary cost. These measures make explicit the trade-off between semantic verification and serving efficiency.

What is established and what remains to be tested

This study defines a formal error class and implements an interoperable state schema with category-specific effects of matched state ablations. It also shows that performance degrades sharply when state must be recovered from partially observable text. Together, these results identify situation sensing as a separate engineering and evaluation target rather than treating every failure as retrieval or generation error.

The audited E5 pilot answers a narrower question: on three Gemini models, explicit situation prompting does not improve accuracy over a matched structured prompt and trails focused lexical retrieval overall. Its category-level exception—situation beats retrieval on temporal validity and observer knowledge—is consistent with applicability reasoning mattering when the decisive variable is which claim is valid now or known to this observer. This single-provider, generated-benchmark result is not sufficient to confirm the general LLM claim. Such confirmation still requires the same model, evidence and computation budget across independently authored natural questions; multiple model families; direct, structured, temporal-retrieval, verification-agent and situation conditions; and paired, scenario-clustered inference. The SE0–SE4 maturity model and longer research programme are provided in Supplementary Information.

Limitations and risks

The diagnostic data are generated from a schema shared with the solver, so benchmark-solver circularity is intentional and limits external validity. The item count does not equal the number of statistically independent observations because examples share template families. The E1–E4 baselines are software controls, while E5 remains limited to one provider family and a small natural-language smoke test. Confidence values are rule-derived and do not constitute neural calibration. No human participants, multilingual evaluation, multimodal sensing or deployed-agent study was completed. The archived three-persona agreement run is explicitly synthetic and validates software plumbing only; it cannot replace independent annotation. Earlier quota-truncated free-tier attempts are archived but excluded from scientific results; only the complete paid-tier run that passes the integrity gate enters the manuscript. These limitations are not resolved by additional prose and require new experiments.

Explicit state can also create new attack surfaces. An adversary may manipulate timestamps, source authority, identity, scope or world labels. A centralized state representation may expose sensitive user information or create false confidence in a flawed schema. Situation-engineered systems therefore require provenance, access control, privacy-preserving memory, conflict visibility and independent evaluation. "Explicit" must not be confused with "correct".

Conclusion

Situation engineering provides a bounded and testable interface between retrieval, state estimation and response policy. The controlled experiments demonstrate that applicability variables have distinct behavioural consequences and that imperfect sensing can erase the advantage of correct update rules. Establishing broad utility now requires the pre-specified natural-text, multi-model comparison against equally resourced structured prompting, temporal retrieval and agent verification.

Methods

Problem formulation

An item consists of a query q, evidence claims E, a query time and, where relevant, an observer or possible-world identifier. A claim may be supported in isolation but inapplicable to the query. We define

$$\mathrm{Apply}(e, q, S) \in \{\mathrm{valid}, \mathrm{invalid}, \mathrm{unresolved}\}, \tag{6}$$

where S is the active situation state. The system first constructs or receives S, then updates competing claims and selects an action from answering, retrieval, clarification, abstention or task-specific action.

The end-to-end system is decomposed as

$$(q, E) \xrightarrow{f_{sense}} \hat{S} \xrightarrow{f_{update}} \hat{S}' \xrightarrow{\pi} a \xrightarrow{g} y. \tag{7}$$

This decomposition supports stage-specific evaluation. The gold-state condition supplies S directly and evaluates f_{update}, π and the deterministic answer renderer. A predicted-state condition applies a text-only deterministic sensor before the same update and policy stages. It evaluates composition on generated claim language, but not f_{sense} on unrestricted natural language.

Relation to prior formalisms

Situation engineering draws on several mature traditions but is not identical to any one of them. Situation semantics treats meaning relative to partial situations [14]. Situation and event calculi formalize how actions change fluents over time [15–17]. Temporal reasoning provides interval relations and validity semantics [18]. Belief revision and truth-maintenance research asks how an agent should update inconsistent

commitments [19, 20]. POMDPs and dialogue-state tracking maintain uncertainty over latent states for sequential decisions [21, 22]. Temporal knowledge graphs and bitemporal databases record changing facts and distinguish when a fact is valid from when it is stored [23].

The proposed contribution is a systems-level applicability contract for generative and agentic AI. It requires these representations to connect to natural-language evidence, provenance, unresolved alternatives and a response policy that can answer, retrieve, clarify, abstain or act. Thus the novelty is not a new universal logic. It is the explicit decomposition of modern AI reliability into (i) sensing a situation from heterogeneous evidence, (ii) maintaining a provenance-linked state, (iii) validating applicability and (iv) propagating unresolved state into behaviour. This framing allows existing logical, probabilistic and neural methods to be compared through a common set of interfaces and failure measurements.

A key distinction is that these traditions vary in whether they jointly expose time, observer, possible world, uncertainty, provenance and an answer-or-action gate; the proposed interface requires all of them to be reportable without claiming that their underlying formalisms are new.

Dynamic epistemic logic and common-ground modelling formalize belief-changing information and mutually recognized conversational commitments [24, 25]. Theory-of-mind models, world models and active perception further address attributed beliefs, latent environment state and information acquisition. These traditions reinforce rather than eliminate the need for careful attribution: situation engineering does not claim their theoretical results as new. Its proposed novelty is the systems contract that exposes their state variables to provenance-aware answer/retrieve/clarify/abstain gates in contemporary generative systems.

Situation schema

The schema contains actors and entities; events and state transitions; valid and observation time; scope; modality and source status; observer-specific knowledge; possible-world identity; provenance; and unresolved answer-critical variables. Update rules include temporal supersession, cancellation, source-authority preference where specified, scope containment, observer visibility and world separation. Unresolved variables are preserved rather than converted into a guessed binary state.

SituationCatch-Bench generation

The deterministic generator creates 600 instances for each of seven categories using seeded lexical variation and shuffled claim order. Each item includes the question, claims, structured situation fields, gold action, gold answer aliases and category. The categories and gold semantics are documented in the bundled datasheet. Generation and evaluation are intentionally co-designed: the benchmark is a conformance suite for explicit semantics, analogous to unit tests for a software protocol, rather than an independent estimate of natural-language performance.

Future benchmark versions should use separate authors for data construction and system development, clustered scenario families, hidden generators, natural and

semi-synthetic sources, adjudicated annotations, compositional splits and adversarial state manipulations. Recommended natural-data fields and annotation instructions are supplied in the Supplementary Information.

Reference implementation

SCQA is a deterministic Python reference implementation with no trained parameters. It parses the supplied structured fields, performs category-specific state updates and selects a permitted response. The three software-control baselines select claims using lexical overlap, recency or latest mention and do not maintain the complete situation state. These controls test whether the specified state variables are necessary for the generated invariants; they are not intended to represent competitive LLM, RAG or agent baselines.

Predicted- and corrupted-state evaluation

The predicted-state sensor receives only claim text and infers event type, actor, temporal expression, modality, source status and world or scope using documented deterministic patterns; it cannot copy hidden gold slots. The downstream engine is unchanged. In a pre-specified corruption condition, predicted source status is replaced by a generic reported source, modality is forced to confirmed and scope is forced to global. All three conditions contain the same 4,200 identifiers and are scored by exact action and answer. Item-level and aggregate outputs are released in `results/state_conditions.csv` and `results/state_conditions_summary.csv`. The sensor is additionally scored claim by claim for actor, event, temporal expression, validity interval, scope, modality, joint source/status, observer-knowledge and world slots. Because claim order is preserved, this comparison does not require learned alignment. The resulting files are `results/predicted_state_slots.csv` and its aggregate summary. We distinguish an *evidence-complete* condition, in which every required slot is supplied, from a *partially observable* condition, in which absent slots must remain unresolved. A text-observable condition must instead render every answer-critical variable in independently authored prose and must not use hidden metadata during sensing. Results should be stratified by these regimes rather than averaging unextractable and extractable slots.

Metrics and analysis

The primary diagnostic measure is exact action-and-answer accuracy. For items requiring clarification, correctness requires selecting clarification rather than providing the latent answer. Category-level accuracy and matched component ablations localize failure mechanisms. Because instances share template families, item-level confidence intervals and P values are not used as primary scientific evidence in the revised manuscript. The released scripts retain item-level summaries for regression testing, but future natural-language studies should use scenario- or template-clustered bootstrap intervals, paired tests, multiple-comparison control and pre-specified primary outcomes.

Rule-derived confidence values from the original implementation are not treated as neural calibration. End-to-end work should report Brier score, expected calibration error, selective risk, risk–coverage area, clarification utility and abstention precision/recall using model- or state-derived probabilities.

Ablation design

Seven ablations independently remove temporal validity, source status, modality, scope, observer knowledge, world identity or missing-variable detection. Each ablation is applied to all items, and its expected effect is specified before execution. The purpose is to test causal alignment between representation and failure category, not to estimate an average treatment effect in natural text.

E5 multi-model pilot and confirmatory protocol

The reported E5 pilot uses a frozen stratified sample of 210 items (30 per category; sampling seed 20260718) and three Gemini-family model snapshots: `gemini-2.5-flash`, `gemini-2.5-flash-lite` and `gemini-3.5-flash`. Each model receives identical evidence under direct, structured and explicit-situation prompts, with temperature and thinking budget set to zero and a common output-token limit. A separate retrieval condition selects the top three claims by token overlap with the question and supplies their plain text to the direct-answer prompt. A tool-using agent condition follows a ReAct-style loop: the model may issue `search` queries over the item's claims, receives the matching claims, and then answers. Every request, raw response, normalized answer, token count, latency, failure and retry is retained. The completion gate joins records to the frozen sample, rejects missing or unexpected identifiers and requires 30 successful responses in every category for every requested model and condition. The reported run contains no failed calls and passes this gate. Stored raw responses were rescored deterministically after the action-equivalence rule was corrected to credit CLARIFY/ABSTAIN uniformly in plain text and JSON; rescoring made no provider calls. The plain prompt elicits the CLARIFY token more readily than the JSON conditions, which accounts for the large hidden-premise differences in Extended Data Table 2.

Accuracy contrasts use category-clustered bootstrap intervals (3,000 resamples, seed 20260726), treating the seven template categories as clusters. These intervals quantify sensitivity across the available template families, not population uncertainty over unrestricted questions. Cost summaries report provider token counts and measured request latency by condition. The bundled 12-item SituatedQA temporal sample is reported only as a smoke test; its size does not support a standalone inferential claim. Only the two date-anchored conditions (structured and situation) are compared there, because the direct prompt omits the query date and is therefore not a matched control. The frozen manifest, model snapshots and cost accounting are in `code/experiment_manifest.e5.json`, and the audited run is reproduced with `python code/run_e5.py`.

The following larger confirmatory study is specified but has not been executed. It would compare these matched conditions: direct prompting; chain-of-thought or structured prompting; date-aware context; standard RAG; temporal or event-aware RAG; reflection or verification agent; and Situation Engineering. The generation model, evidence set, retrieval corpus, maximum context, output budget and sampling parameters should be held constant where the design permits. The minimum study uses two proprietary and two strong open-weight model families with immutable model identifiers; additional model families test robustness rather than substitute for balanced completion.

The primary intervention contrast must be *same model + same evidence + same computation budget* with versus without the explicit situation layer. A factorial analysis should independently vary prompt, context/retrieval, harness and situation state. Competitive conditions should include long-context prompting, few-shot structured reasoning, self-consistency, self-refinement, Self-RAG, graph or temporal retrieval, dialogue/belief-state tracking, clarification and calibrated abstention.

For each call, researchers should archive the prompt, evidence, raw response, normalized answer, action decision, model identifier, provider, execution date, latency, token use, monetary cost, errors and retries. Evaluation must report gold-state, predicted-state and deliberately corrupted-state conditions. The predicted-state condition should score entity/event extraction, temporal relations, validity intervals, scope, modality, source status, observer knowledge, possible-world identity and final answer. This enables error attribution to sensing, updating, policy or generation.

The natural bridge must include at least one independently constructed temporal or situated benchmark (for example SituatedQA, FreshQA or ChronoQA), with frozen item inclusion and scoring rules. Synthetic diagnostic, natural benchmark and model intervention are reported as separate evidence levels.

The bundled `code/run_multimodel_eval.py` provides provider adapters and refuses to create placeholder outputs when credentials are absent. Earlier quota-truncated free-tier attempts remain archived but are excluded from every scientific aggregate. Only the complete paid-tier records admitted by `code/audit_e5.py` enter the reported E5 tables. The completed pilot is evidence from one provider family, not the multi-family confirmatory experiment.

Prospective test designed to break benchmark–solver coupling

We pre-specify the next test rather than presenting simulated numbers as a completed experiment. An independent data team should sample naturally occurring questions from at least three domains, write evidence packs without access to the SCQA schema or solver, and freeze them before the systems team maps evidence into states. The crossed evaluation uses at least three genuinely independent model families (including an open-weight family), with identical evidence and matched token and tool budgets across direct, structured, temporal-retrieval, verification-agent and situation conditions. Items are split by scenario—not paraphrase—and held-out domains and generator authors are reported separately. Three blinded human annotators label answerability, answer, applicable evidence and required state slots; disagreements are adjudicated only after independent labels are frozen.

Supplementary Note 14 records the pre-specified strata, collection targets and split rules; it reports no outcome values, because none have been measured. The confirmatory claim would require a positive situation-versus-structured paired contrast whose scenario-clustered 95% interval excludes zero in at least two independent families, no material loss against temporal retrieval, and replication on the author-disjoint natural set. Human agreement, selective risk, latency, tokens and cost must be reported regardless of outcome. Until those gates are passed, generator–solver co-design, one provider family and the absence of human evaluation remain core limitations rather than prose-resolved concerns.

Generalization and efficiency protocol

External evaluation should pre-register held-out scenario families and test unseen templates, longer event chains, nested scopes, conflicting authoritative sources, multiple observers, multilingual questions and adversarial stale evidence. Splits must occur at scenario or generator-grammar level rather than over lexical variants. Uncertainty intervals should use scenario-clustered resampling. Each condition must report latency, input and output tokens, retrieval and tool calls, retries, monetary cost and clarification overhead. Accuracy gains without matched resource reporting cannot identify the effect of the situation layer.

Human and deployment evaluation

Human evaluation should use at least three independent annotators per natural item, blinded to method, with a documented adjudication procedure. Recommended outcomes include state-slot agreement, answer correctness, appropriateness of clarification or abstention, unnecessary clarification and perceived justification quality. Studies involving participants should report ethics approval or exemption, consent, compensation and privacy handling.

The released annotation program generates separately shuffled, blinded packets for three or more annotators. It requires complete independent labels before computing Fleiss' κ and unanimous agreement for action, answer and state slots; incomplete packets terminate with an error. Disagreements are adjudicated only after independent files are frozen. Blank blinded packets for a 70-item stratified subset are released under `paper/annotation_packets/`. No human agreement value is reported because independent human annotations were not collected. For software testing, a separate command generates three deterministic persona packets with injected disagreements. Its manifest and score output are marked `synthetic: true` and "NOT HUMAN-SUBJECT EVIDENCE"; those values are excluded from the manuscript's empirical claims.

Deployment studies should additionally measure state drift, invalidation latency, recovery after contradictory observations, action-precondition violations, security attacks on state metadata, throughput, tail latency, energy, token use and cost. The state and provenance required to audit a consequential decision should be retained subject to privacy and access-control requirements.

Reproducibility and reporting protocol

The package contains the generator, generated benchmark, reference implementation, predictions, figures, datasheet and exact commands. To make future systems comparable, authors should report: (1) the situation schema; (2) which variables are supplied, retrieved or inferred; (3) update and conflict semantics; (4) response and action policies; (5) provenance and observability; (6) stage-specific errors; (7) prompt, context, retrieval, memory, tool and harness configurations; and (8) resource costs. A machine-readable experiment manifest is recommended.

Ethics and use of generative AI

The controlled benchmark contains no personal records or human participants. Generative AI assisted drafting and software development. The author inspected the code, generated outputs, calculations and references and remains responsible for the manuscript. No empirical values were fabricated to represent unexecuted model calls or human studies.

Data availability

SituationCatch-Bench v0.1 is included in the public source repository at https://github.com/leemgs/sage and in the submission package as `paper/data/situationcatch_bench.jsonl`, together with its deterministic generator, datasheet and item-level outputs. This package is sufficient to reproduce the controlled results. The E5 and SituatedQA item-level raw responses are under `paper/results/raw*`. The exact reviewed version is identified by the Git commit or release tag supplied with the submission. Repository-owned materials are distributed under the MIT licence; third-party data retain their source licences. No unminted DOI is represented as an existing deposit.

Code availability

The complete executable implementation is included under `code/`. Running `PYTHONPATH=code python code/run_experiments.py` regenerates gold-, predicted- and corrupted-state outputs without network access or proprietary dependencies. The multi-model adapter records raw responses and errors and never creates synthetic API results. The annotation program requires three complete independent packets before reporting agreement and labels simulated-persona runs as synthetic rather than human evidence. Source and item-level outputs are also available at https://github.com/leemgs/sage. The repository README records exact commands; `SHA256SUMS.txt` records packaged artefact checksums. A public release tag should identify the reviewed version; no DOI is claimed unless an archive deposit has actually been minted.

Author Contributions

G.L.: Conceptualization, Formal analysis, Investigation, Methodology, Software, Validation, Visualization, Writing – original draft, Writing – review & editing.

Competing Interests

The author declares no competing interests.

Acknowledgements

The author received no specific funding for this work.

References

[1] Zhang, M. J. Q. & Choi, E. SituatedQA: incorporating extra-linguistic contexts into QA. In *Proc. EMNLP*, 7371–7387 (2021).

[2] Vu, T. et al. FreshLLMs: refreshing large language models with search engine augmentation. In *Findings of the Association for Computational Linguistics: ACL 2024*, 13697–13720 (2024).

[3] Meem, J., Rashid, M., Dong, Y. & Hristidis, V. PAT-Questions: a self-updating benchmark for present-anchored temporal question-answering. In *Findings of the Association for Computational Linguistics: ACL 2024*, 13129–13148 (2024).

[4] Chen, W., Wang, X. & Wang, W. Y. A dataset for answering time-sensitive questions. In *Proc. Neural Information Processing Systems Track on Datasets and Benchmarks* **1** (2021).

[5] Chen, Z. et al. A question answering dataset for temporal-sensitive retrieval-augmented generation (ChronoQA). *Sci. Data* **12**, 1855 (2025).

[6] Lewis, P. et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Adv. Neural Inf. Process. Syst.* **33**, 9459–9474 (2020).

[7] Asai, A., Wu, Z., Wang, Y., Sil, A. & Hajishirzi, H. Self-RAG: learning to retrieve, generate, and critique through self-reflection. In *Proc. International Conference on Learning Representations* (2024).

[8] Yao, S. et al. ReAct: synergizing reasoning and acting in language models. In *Proc. International Conference on Learning Representations* (2023).

[9] Schick, T. et al. Toolformer: language models can teach themselves to use tools. *Adv. Neural Inf. Process. Syst.* **36**, 68539–68551 (2023).

[10] Mei, L. et al. A survey of context engineering for large language models. *Preprint at* arXiv (2025).

[11] Zhong, H. & Zhu, S. AI harness engineering: a runtime substrate for foundation-model software agents. *Preprint at* arXiv (2026).

[12] Lin, J. et al. Agentic harness engineering: observability-driven automatic evolution of coding-agent harnesses. *Preprint at* arXiv (2026).

[13] Kim, S., Wang, J., Xie, X. & Whang, S. E. Harnessing temporal databases for systematic evaluation of factual time-sensitive question answering in large language models. In *Proc. International Conference on Learning Representations* (2026).

[14] Barwise, J. & Perry, J. *Situations and Attitudes* (MIT Press, 1983).
[15] McCarthy, J. & Hayes, P. J. Some philosophical problems from the standpoint of artificial intelligence. In *Machine Intelligence 4* (eds Meltzer, B. & Michie, D.) 463–502 (Edinburgh Univ. Press, 1969).
[16] Kowalski, R. & Sergot, M. A logic-based calculus of events. *New Gener. Comput.* **4**, 67–95 (1986).
[17] Reiter, R. *Knowledge in Action: Logical Foundations for Specifying and Implementing Dynamical Systems* (MIT Press, 2001).
[18] Allen, J. F. Maintaining knowledge about temporal intervals. *Commun. ACM* **26**, 832–843 (1983).
[19] Alchourrón, C. E., Gärdenfors, P. & Makinson, D. On the logic of theory change: partial meet contraction and revision functions. *J. Symb. Logic* **50**, 510–530 (1985).
[20] Doyle, J. A truth maintenance system. *Artif. Intell.* **12**, 231–272 (1979).
[21] Kaelbling, L. P., Littman, M. L. & Cassandra, A. R. Planning and acting in partially observable stochastic domains. *Artif. Intell.* **101**, 99–134 (1998).
[22] Henderson, M., Thomson, B. & Williams, J. D. The second dialog state tracking challenge. In *Proc. SIGDIAL*, 263–272 (2014).
[23] Snodgrass, R. T. Temporal databases. In *Theories and Methods of Spatio-Temporal Reasoning in Geographic Space* (eds Frank, A. U., Campari, I. & Formentini, U.) Lecture Notes in Computer Science Vol. 639, 22–64 (Springer, 1992).
[24] van Benthem, J. Dynamic logic for belief revision. *J. Appl. Non-Classical Logics* **17**, 129–155 (2007).
[25] Stalnaker, R. Common ground. *Linguist. Philos.* **25**, 701–721 (2002).

Extended Data

Extended Data Table 1 Claim-level accuracy of the deterministic text sensor ($n = 7{,}800$ claims per slot).

Slot	Actor	Event	Time	Validity	Scope	Modality	Source	Observer	World
Accuracy (%)	30.8	92.3	0.0	0.0	61.5	100.0	30.8	92.3	100.0

Extended Data Table 2 Retrieval-augmented (RAG), full-evidence prompting, and a tool-using agent, mean accuracy over the three Gemini models on the $n = 210$ sample. RAG answers from the top-3 lexically retrieved claims; Structured and Situation see all claims; Agent may issue its own search queries over the item's claims before answering. Descriptively, on the two categories that most require reasoning over the applicability state – temporal validity and observer knowledge – the full-evidence conditions score above both retrieval and the agent; this category-level pattern is exploratory, and none of the retrieval-, prompt-, or tool-based interventions dominates overall.

Category	RAG (top-3)	Structured	Situation	Agent
Temporal	91.1	100.0	100.0	64.4
Observer	81.1	84.4	86.7	66.7
Counterfact.	100.0	100.0	85.6	91.1
Scope	100.0	100.0	93.3	86.7
Source confl.	100.0	100.0	95.6	96.7
Modality	100.0	100.0	92.2	100.0
Hidden prem.	66.7	0.0	11.1	36.7
Overall	91.3	83.5	80.6	77.5